

REPORT

Hệ điều hành thời gian thực RTOS và ứng dụng lập trình đa nhiệm cho Drone

Thực hiện: Nguyễn Quốc Anh

CHƯƠNG 1 Tìm hiểu về Drone

1.1. Cảm biến con quay hồi chuyển và gia tốc góc nghiêng

1.1.1. Giới thiệu cảm biến MPU6050

MPU6050 là cảm biến được lựa chọn do có độ chính xác cao, giá thành hợp lý, dễ tiếp cận.

IC MPU6050 được tích hợp con quay hồi chuyển (gyroscope) và cảm biến gia tốc (accelerometer) 3 trục

Cảm biến gia tốc có nhiệm vụ đo gia tốc tác dụng lên cảm biến theo 3 phương X, Y, Z và trả về dữ liệu thô 16 bit, dữ liệu này sau đó được chia cho thang đo để lấy giá trị của gia tốc (đơn vị g – gia tốc trọng trường), $g = 9.78 - 9.83 \text{ m/s}^2$, $g = \frac{GM}{(R+h)^2}$

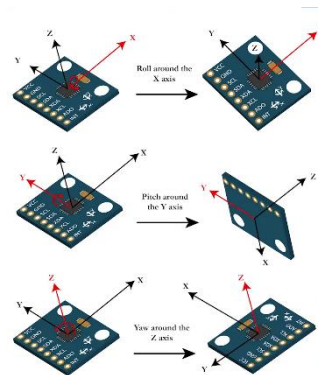
VD: một vật di chuyển theo phương X, chiều di chuyển là chiều dương của trục với gia tốc $a = 4,5 \text{ m/s}^2$, khi đó, gia tốc của vật bao gồm

$$A_x = 4,5/g = 0.46 \text{ g}$$

$$A_y = 0$$

$$A_z = 1 \text{ g}$$

Cảm biến con quay hồi chuyển trả về dữ liệu 16 bit, từ dữ liệu này có thể tính toán ra vận tốc góc roll, pitch, yaw (quanh trục X, quanh trục Y, quanh trục Z) bằng cách lấy dữ liệu thô chia (65535/thang quy đổi ($^{\circ}/\text{s}$))



1.1.2. Hiệu chỉnh cảm biến

Cảm biến con quay hồi chuyển luôn có sai số offset nhất định (vận tốc góc không xấp xỉ 0 dù cảm biến đứng yên, do nhiễu điện từ, drift, nhiệt độ...); vì vậy, để đảm bảo phép đo chính xác, cần có bước hiệu chỉnh cảm biến trước khi hệ thống hoạt động.

Để loại bỏ sai số offset, thực hiện lấy mẫu tín hiệu 2000 - 5000 lần từ cảm biến MPU trong trạng thái đứng yên, lấy trung bình các mẫu tín hiệu và đặt làm offset.

```
void MPU_calibrate(float *gyro_offset)
{
    Gx -= gyro_offset[0];
    Gy -= gyro_offset[1];
    Gz -= gyro_offset[3];
}

    current_tick = HAL_GetTick();
    while(sample++ != number_of_sample)
    {
        MPU_Read_Accel();
        MPU_Read_Gyro();
        gyro_offset[0] += Gx;
        gyro_offset[1] += Gy;
        gyro_offset[2] += Gz;
        HAL_Delay(1);
    }
    for(int i = 0; i<3 ; i++)
    {
// get offset
        gyro_offset[i] /=(float)(sample-1);
    }
    current_tick = HAL_GetTick() - current_tick;
    MPU_calibrate((float*)gyro_offset);
void MPU_calibrate(float *gyro_offset)
{
    Gx -= gyro_offset[0];
    Gy -= gyro_offset[1];
    Gz -= gyro_offset[3];
}
```

Độ chính xác, độ trễ phép hiệu chỉnh (ms), offset trên 2 cảm biến (MPU6050 và MPU 92.65=> con này rởm) khác nhau:

⌘= Gx	float	-3.43511462
⌘= Gy	float	1.3358779
⌘= Gz	float	0.480916023
▼ 📦 gyro_offset	volatile float [3]	[3]
⌘= gyro_offset[0]	volatile float	-3.71429515
⌘= gyro_offset[1]	volatile float	1.33873081
⌘= gyro_offset[2]	volatile float	0.493888885
⌘= sample	uint16_t	2001
⌘= current_tick	uint32_t	6000

⌘= Gx	float	-0.0644464493
⌘= Gy	float	-0.132428408
⌘= Gz	float	0.164003909
▼ 📦 gyro_offset	volatile float [3]	[3]
⌘= gyro_offset[0]	volatile float	-3.6912787
⌘= gyro_offset[1]	volatile float	1.33090174
⌘= gyro_offset[2]	volatile float	0.492484629
⌘= sample	uint16_t	2001
⌘= current_tick	uint32_t	6000

Với 5000 lần lấy mẫu và hiệu chỉnh tổng mất khoảng 15000ms – tương đối dài do khối lượng tính toán rất lớn khi vi điều khiển thao tác với số thực.

⌘= Gx	float	2.32061076
⌘= Gy	float	1.50381684
⌘= Gz	float	-1.75572515
▼ 📦 gyro_offset	volatile float [3]	[3]
⌘= gyro_offset[0]	volatile float	2.34823799
⌘= gyro_offset[1]	volatile float	1.5399605
⌘= gyro_offset[2]	volatile float	-1.69074082
⌘= sample	uint16_t	5001
⌘= current_tick	uint32_t	15000

⌘= Gx	float	-0.0276272297
⌘= Gy	float	-0.0361436605
⌘= Gz	float	-0.0649843216
▼ 📦 gyro_offset	volatile float [3]	[3]
⌘= gyro_offset[0]	volatile float	2.34823799
⌘= gyro_offset[1]	volatile float	1.5399605
⌘= gyro_offset[2]	volatile float	-1.69074082
⌘= sample	uint16_t	5001
⌘= current_tick	uint32_t	15000

Hiệu chỉnh cảm biến gia tốc:

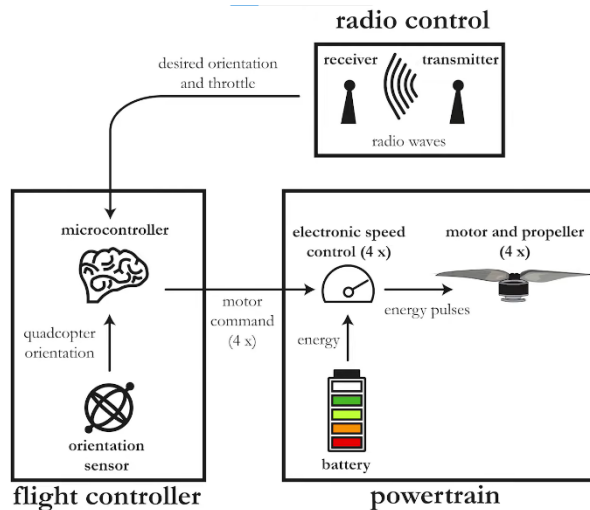
1.2. GPS

Module NEO

1.3. Module la bàn điện từ

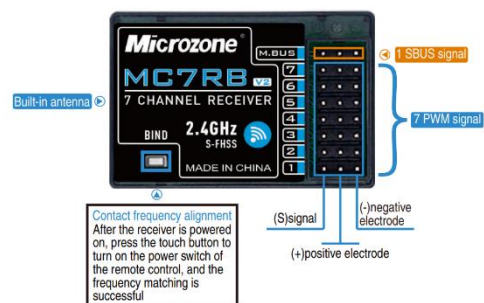
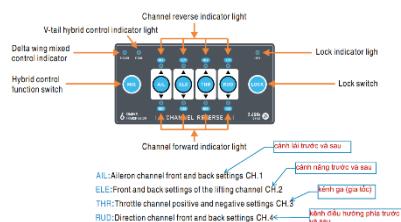
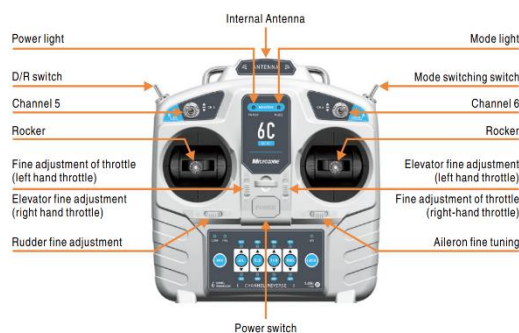
Module HMC 5883

CHƯƠNG 2 TRANMISTER VÀ RECEIVER



Tranmister là một bộ điều khiển cho phép người dùng truyền tín hiệu điều khiển vô tuyến tới drone, receiver được tích hợp trên drone nhận tín hiệu và giải mã, sau đó tín hiệu được mã hóa và truyền tới vi điều khiển thông qua một giao thức có dây.

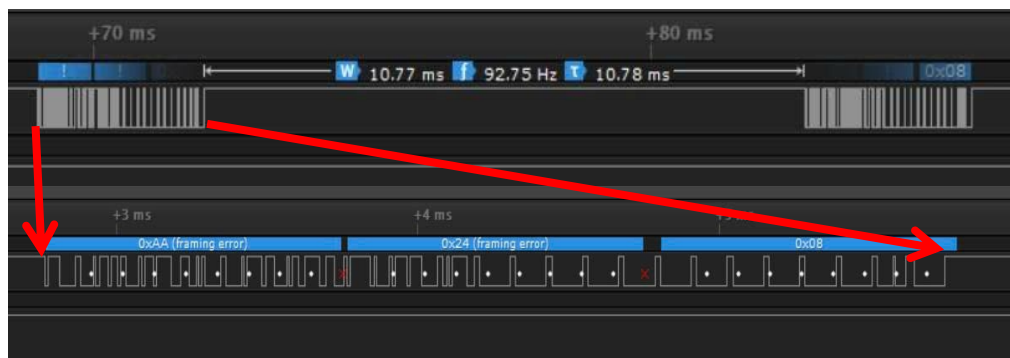
2.1. Lựa chọn transmitter và receiver



2.2. Đọc tín hiệu từ receiver gửi tới MCU

- SBUS protocol: SBUS là giao thức được phát triển từ RS232, thường dùng cho điều khiển servo. Khác với PWM, SBUS có thể giao tiếp với tối đa 16 servo mà chỉ cần 1 dây truyền duy nhất.
- SBUS dùng inverted serial logic với baudrate 100000, 1 frame truyền gồm 8 data bits, even parity bit và 2 stop bits. Một packet gồm 25 bytes dữ liệu:
- Byte[0]: SBUS header, 0x0F
- Byte[1 -22]: 16 servo channels, 11 bits each
- Byte[23]
 - o Bit 0: channel 17 (0x01)
 - o Bit 1: channel 18 (0x02)
 - o Bit 2: frame lost (0x04)
 - o Bit 3: failsafe activated (0x08)
- Byte[24]: SBUS footer

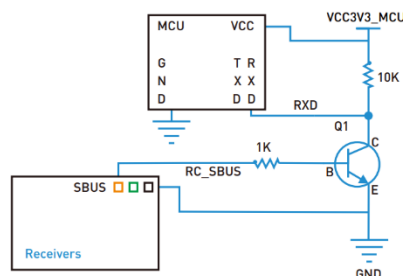
Khung truyền tín hiệu được gửi mỗi 14ms, thời gian truyền khoảng 3 – 4ms.



Với MC7RE, bit 4 của flag báo hiệu mất kết nối với transmitter, và chỉ sử dụng tối đa 6 kênh, mỗi kênh 11 bit => 9 bytes data đầu tiên của packet có dữ liệu của 6 kênh. Bên cạnh đó, cần lắp mạch đảo tín hiệu trước khi kết nối với MCU.

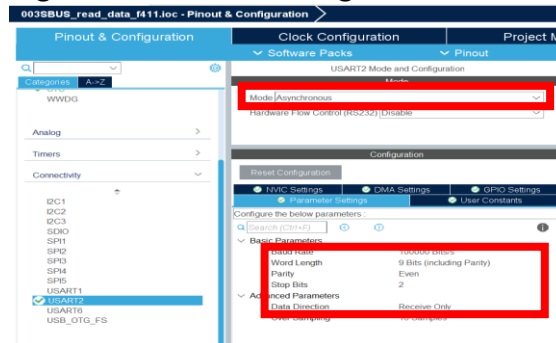
SBUS signal:

High level 3.3V, 100k baud rate, 8 data bits (9 bits to be selected in stm32), even parity (EVEN), 2 stop bits, starting at 0x0F and ending at 0x00, 25 bytes in total, one frame of data every 14ms;
arranged as follows: [0x0F] [data0] [data1] [data2] [data3] ... [data10] [flag] [0x00];
data1.....data22: Corresponding to 16 channels (ch1-ch16), each channel 11bit, channel data value range is 200-1800, docking SBUS signal needs to invert the level in hardware, software inversion is invalid, inversion circuit is as follows.



Theo reference manual, cấu hình UART2 cho MCU như sau:

Cho phép ngắt UART2, generate code, sử dụng các API sau để kiểm tra MCU có



nhận được tín hiệu hay không:

```
HAL_StatusTypeDef HAL_UART_Receive_IT(UART_HandleTypeDef *huart, uint8_t *pData,
uint16_t Size) // khởi động chế độ nhận dữ liệu có tín hiệu ngắt nhận
```

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) // hàm xử lý callback
ngắt uart
```

UART2_RX	uint8_t [25]	16-bit unsigned integer
sbusrx[0]	uint8_t	15 '\017'
sbusrx[1]	uint8_t	192 'À'
sbusrx[2]	uint8_t	116 'Y'
sbusrx[3]	uint8_t	89 'Y'
sbusrx[4]	uint8_t	56 '8'
sbusrx[5]	uint8_t	198 'Z'
sbusrx[6]	uint8_t	135 '\207'
sbusrx[7]	uint8_t	12 'f'
sbusrx[8]	uint8_t	100 'd'
sbusrx[9]	uint8_t	32 '-'
sbusrx[10]	uint8_t	3 '\003'
sbusrx[11]	uint8_t	25 '\031'
sbusrx[12]	uint8_t	0 '\0'
sbusrx[13]	uint8_t	0 '\0'
sbusrx[14]	uint8_t	0 '\0'
sbusrx[15]	uint8_t	0 '\0'
sbusrx[16]	uint8_t	0 '\000'
sbusrx[17]	uint8_t	0 '\000'
sbusrx[18]	uint8_t	0 '\000'
sbusrx[19]	uint8_t	0 '\000'
sbusrx[20]	uint8_t	0 '\000'
sbusrx[21]	uint8_t	0 '\000'
sbusrx[22]	uint8_t	0 '\0'
sbusrx[23]	uint8_t	0 '\0'
sbusrx[24]	uint8_t	0 '\0'

Hình 1 tín hiệu nhận từ receiver khi kết nối thành công với transmitter

Expression	Type	Value
current_tick		Failed to evaluate expression
sbusrx	uint8_t [25]	[25]
sbusrx[0]	uint8_t	15 '\017'
sbusrx[1]	uint8_t	232 'è'
sbusrx[2]	uint8_t	67 'C'
sbusrx[3]	uint8_t	31 '\037'
sbusrx[4]	uint8_t	10 '\n'
sbusrx[5]	uint8_t	208 'ð'
sbusrx[6]	uint8_t	135 '\207'
sbusrx[7]	uint8_t	62 '>'
sbusrx[8]	uint8_t	244 'ô'
sbusrx[9]	uint8_t	161 'í'
sbusrx[10]	uint8_t	15 '\017'
sbusrx[11]	uint8_t	125 '}
sbusrx[12]	uint8_t	0 '\0'
sbusrx[13]	uint8_t	0 '\0'
sbusrx[14]	uint8_t	0 '\0'
sbusrx[15]	uint8_t	0 '\0'
sbusrx[16]	uint8_t	0 '\000'
sbusrx[17]	uint8_t	0 '\000'
sbusrx[18]	uint8_t	0 '\000'
sbusrx[19]	uint8_t	0 '\000'
sbusrx[20]	uint8_t	0 '\000'
sbusrx[21]	uint8_t	0 '\000'
sbusrx[22]	uint8_t	0 '\0'
sbusrx[23]	uint8_t	16 '\020'
sbusrx[24]	uint8_t	0 '\0'

Hình 2 tín hiệu nhận từ receiver khi mất kết nối thành công với transmitter

Vấn đề đồng bộ khung truyền: ngoại vi UART chỉ phát hiện được frame truyền theo định dạng đã cấu hình sẵn nhưng không thể nhận biết khi nào là bắt đầu của packet => cần quá trình đồng bộ khung truyền.

2.1. DMA UART

CHƯƠNG 3 ĐIỀU KHIỂN ĐỘNG CƠ BLDC

CHƯƠNG 4